



Chuck Norris Jokes

Documentação Completa do Projeto iOS



Visão Geral

Aplicativo iOS desenvolvido em **Swift** usando **UIKit** e **ViewCode**, que consome a API pública Chuck Norris para exibir piadas aleatórias e categorizadas.



Status: Projeto Completo

Todos os 10 requisitos mínimos foram atendidos com sucesso.



Requisitos Atendidos

Requisitos Técnicos

- ✓ Swift - Linguagem principal
- ✓ UIKit - Framework de interface
- ✓ ViewCode - Sem Storyboard, tudo em código
- ✓ MVVM - Arquitetura Model-View-ViewModel
- ✓ URLSession - Requisições HTTP
- ✓ Codable/Decodable - Parse de JSON
- ✓ Auto Layout - Constraints programáticas

Requisitos Mínimos (10/10)

- ✓ Desenvolvido utilizando ViewCode
- ✓ Utiliza arquitetura MVVM
- ✓ Consome a API utilizando URLSession
- ✓ Utiliza Codable/Decodable para parse de JSON
- ✓ Busca uma piada ao iniciar o aplicativo
- ✓ Permite buscar uma nova piada
- ✓ Exibe Loading durante uma requisição
- ✓ Trata possíveis erros
- ✓ Possui uma tela com as categorias disponíveis
- ✓ Permite selecionar uma categoria e buscar uma piada correspondente



Estrutura do Projeto

```
Chuck Norris Jokes/  
├── Models/  
│   └── Joke.swift (Decodable)  
├── Services/  
│   └── JokeService.swift (URLSession + Protocol)  
├── Scenes/  
│   ├── JokeView.swift (ViewCode)  
│   ├── JokeViewController.swift (Controle)  
│   ├── JokeViewModel.swift (Lógica)  
│   └── Categorias/  
│       ├── CategoriesView.swift (TableView)  
│       ├── CategoriesViewController.swift  
│       └── CategoriesViewModel.swift  
├── SceneDelegate.swift (TabBar Navigation)  
└── AppDelegate.swift (não modificado)
```



Detalhes de Cada Arquivo



Models/Joke.swift

Objetivo: Estrutura de dados que representa uma piada da API

Características:

- ✓ **struct Joke: Decodable** - Converte JSON automaticamente
- ✓ **CodingKeys** - Mapeia snake_case (JSON) → camelCase (Swift)
- ✓ Extension com propriedade **imageUrl** - Converte string em URL
- ✓ Comentários em cada propriedade

Exemplo JSON:

```
{
  "id": "abc123",
  "value": "Chuck Norris can divide by zero.",
  "icon_url": "https://...",
  "url": "https://...",
  "created_at": "2020-01-05",
  "updated_at": "2020-01-05",
  "categories": []
}
```

2 Services/JokeService.swift

Objetivo: Isolar comunicação com a API

Componentes:

a) NetworkError (enum)

```
enum NetworkError: Error {
    case invalidURL
    case noData
    case decodingError
    case httpError(statusCode: Int)
    case unknownError(Error)
}
```

b) JokeServiceProtocol

- ✓ Define contrato de serviço
- ✓ Permite mock/testing
- ✓ Comentários em cada função

c) JokeService (Implementação)

- ✓ `fetchRandomJoke()` - GET `/jokes/random`
- ✓ `fetchJokeByCategory()` - GET `/jokes/random?category=X`
- ✓ `fetchCategories()` - GET `/jokes/categories`
- ✓ `performRequest()` - Função genérica universal

Mapeamento de Erros:

Condição	NetworkError
URL malformada	<code>.invalidURL</code>
Sem conexão	<code>.unknownError</code>
Status 404, 500	<code>.httpError(statusCode)</code>
JSON inválido	<code>.decodingError</code>
Resposta vazia	<code>.noData</code>

3 Scenes/JokeViewModel.swift

Objetivo: Lógica de negócio e gerenciamento de estado

Estados Possíveis:

```
enum ViewState {  
    case loading                // Carregando  
    case success(joke: Joke)    // Piada carregada
```

```
case error(message: String) // Erro com mensagem
}
```

Métodos Públicos:

Método	O quê	Quando
<code>loadRandomJoke()</code>	Busca piada aleatória	Ao abrir app ou clicar botão
<code>loadJokeByCategory()</code>	Busca piada de categoria	Ao selecionar categoria
<code>retry()</code>	Tenta novamente	Ao clicar "Tentar Novamente"

Observador:

```
var onChanged: (() -> Void)?
```

Avisa ViewController quando estado muda. `DispatchQueue.main.async` garante atualização na main thread.

4 Scenes/JokeView.swift

Objetivo: Interface visual em ViewCode

Componentes (lazy var):

Componente	Tipo	Função
<code>imageView</code>	UIImageView	Exibe ícone Chuck Norris
<code>loadingIndicator</code>	UIActivityIndicatorView	Spinner de carregamento
<code>jokeLabel</code>	UILabel	Texto da piada
<code>errorLabel</code>	UILabel	Mensagem de erro

newJokeButton

UIButton

Botão "Nova Piada"

Layout (Constraints):

- ✓ Todas as constraints em UMA chamada `NSLayoutConstraint.activate([])`
- ✓ Safe area layout
- ✓ Componentes centralizados com padding 16pt
- ✓ Botão 50pt de altura

5 Scenes/JokeViewController.swift

Objetivo: Conectar View com ViewModel

Fluxo:

1. `loadView()` - Define jokeView como view principal
2. `viewDidLoad()` - Configura delegates e observers
3. `updateUI()` - Responde aos 3 estados (loading, success, error)
4. Implementa `JokeViewDelegate` - Quando botão clicado

Carregamento de Imagem:

```
URLSession.shared.dataTask(with: url) { data, _, _ in
    if let data = data, let image = UIImage(data: data) {
        DispatchQueue.main.async {
            self?.jokeView.imageView.image = image
        }
    }
}.resume()
```

6 SceneDelegate.swift

Objetivo: Configurar navegação principal com TabBar

Estrutura:

```
TabBarController
├── [0] JokeNavigationController
│   └── JokeViewController (Piadas)
├── [1] CategoriesNavigationController
│   └── CategoriesViewController (Categorias)
```

Abas:

Aba	Título	Ícone	Tag
0	"Piadas"	laugh	0
1	"Categorias"	list.bullet	1

Navegação (Categoria → Piada):

```
extension SceneDelegate: CategoriesViewControllerDelegate {
    func didSelectCategory(_ category: String) {
        // 1. Cria novo ViewModel com categoria
        viewModel.loadJokeByCategory(category)
        // 2. Cria novo ViewController
        let jokeViewController = JokeViewController(viewModel: viewModel)
        // 3. Faz push na navigation stack
        navController.pushViewController(jokeViewController, animated: true)
        // 4. Volta para aba 0 (Piadas)
        tabBar.selectedIndex = 0
    }
}
```

Fluxo MVVM

Ao Abrir o App

1. SceneDelegate cria JokeService (compartilhado)
2. SceneDelegate cria JokeViewController com ViewModel
3. ViewController conecta delegate e observer
4. ViewController chama viewModel.loadRandomJoke()
5. ViewModel muda estado para .loading

6. Service faz GET /jokes/random
7. Resposta decodificada → ViewModel.success
8. Observer notifica ViewController
9. ViewController atualiza UI

Ao Selecionar Categoria

1. Usuário toca em categoria na TableView
2. CategoriesViewController chama delegate.didSelectCategory()
3. SceneDelegate recebe notificação
4. SceneDelegate cria novo JokeViewController
5. Faz push na navigation stack
6. Volta para aba "Piadas"
7. Novo ViewController mostra piada da categoria

Tratamento de Estados

Tela de Piadas

Estado	UI Mostrada
Loading	Spinner no centro
Success	Imagem + Texto + Botão "Nova Piada"
Error	Mensagem vermelha + Botão "Tentar Novamente"

Tela de Categorias

Estado	UI Mostrada
Loading	Spinner no centro
Success	TableView com categorias selecionáveis
Error	(Esconde tudo)



Arquitetura MVVM

Separação de Responsabilidades

Camada	Responsabilidade
Model	Estrutura de dados (Joke)
View	Desenhar interface (JokeView, CategoriesView)
ViewController	Conectar View com ViewModel
ViewModel	Lógica de negócio + gerenciamento de estado
Service	Comunicação com API

Fluxo de Dependências

```
View
  ↓ (informa eventos)
ViewController
  ↓ (usa)
ViewModel
  ↓ (usa)
Service
  ↓ (usa)
Model
```



Histórico de Commits

```
db61bbc - Completar CategoriesView com constraints
96a867f - Criar CategoriesViewModel
18d9b36 - Criar CategoriesViewController
58870b8 - Criar CategoriesView com TableView
6eea295 - Criar JokeService com URLSession
7c33b3e - Criar Model Joke e JokeViewModel
6b14e88 - Criar JokeViewController com bindViewModel
```

6608179 - Criar JokeView com layout programático
65df029 - Initial commit

✨ Conclusão

Projeto finalizado com sucesso! 🎉

Todos os requisitos foram atendidos com código bem organizado, comentado e seguindo as melhores práticas de arquitetura MVVM.

Documentação gerada em 2026 | Chuck Norris Jokes App